

KONAET OS ENGINEERING DOSSIER

Manual técnico integral

Arquitetura, cadeia de boot, subsistemas, evidências, limites e roteiro completo da fundação atual até um sistema operacional de uso geral.

0.3.4 / Memory Services I / Meridian

x86-64 UEFI · 27 de agosto de 2026

Build/host: 2.435 PASS · VMware M1-M4 + IPC Core I: PASS · Hardware físico: PENDENTE

Documento de engenharia. Não declara sistema de uso geral concluído, superioridade global nem validação em hardware físico.

Sumário

- | | |
|--|---|
| 1. 1. Finalidade e regra de verdade | 11. 11. Build, testes e reprodução |
| 2. 2. Estado comprovado da release | 12. 12. Artefatos auditados |
| 3. 3. Cadeia completa do artefato atual | 13. 13. Validação VMware isolada |
| 4. 4. Cadeia de boot e separação de plataforma | 14. 14. Diagnóstico e recuperação |
| 5. 5. CPU, exceções e serial | 15. 15. Arquitetura de uso geral desejada |
| 6. 6. Interrupções, timer e idle | 16. 16. Gates obrigatórios G00–G18 |
| 7. 7. Memória | 17. 17. Ordem de construção |
| 8. 8. Entrada, Control Center e workspace | 18. 18. Critério de superioridade |
| 9. 9. Intelligence Seed | 19. 19. Definição final de pronto |
| 10. 10. Segurança e modelo de confiança | 20. 20. Mapa de fontes |

1. Finalidade e regra de verdade

O Konaet OS é uma implementação autoral de kernel e ambiente operacional. A meta é construir uma pilha controlada pelo projeto, do boot ao userland, sem incorporar outro sistema operacional no artefato. Imagens conceituais definem a direção visual, mas não comprovam drivers, rede, IA, segurança ou desempenho.

Todo recurso usa um destes estados:

- **PASS:** implementação, gate automatizado e evidência exigida concluídos;
- **IMPLEMENTED / E2E PENDING:** código e contratos existem, mas falta execução física reproduzível;
- **IN PROGRESS:** parte da fundação existe;
- **NOT IMPLEMENTED:** ainda não existe código funcional correspondente.

FULL / END-TO-END READY só pode ser usado quando os gates G00–G18 do capítulo 16 estiverem PASS na mesma release e no mesmo hardware declarado.

2. Estado comprovado da release

Camada	Estado	Evidência atual
Build freestanding	PASS	PE32+ UEFI, warnings como erro, zero imports de host
Empacotamento	PASS	FAT12, ISO9660 e El Torito EFI verificados
Soberania da release	PASS	emulador não obrigatório; nenhum artefato VM ativo
Konaet Boot Protocol v2	PASS em host	ABI de 88 bytes e 32 contratos
Renderer	PASS em host	210 cenários com canários
CPU/descriptores	PASS em host	GDT, IDT, TSS/IST/RSP0, contexto e 59 contratos
Memória física	PASS em host	208 contratos
Heap limitado	PASS em host	37 contratos
Clock monotônico	PASS em host	38 contratos
PIC/PIT/IRQ0/IRQ1	IMPLEMENTED / VM E2E PASS	154 contratos; IRQ0 confirmado no VMware
Teclado PS/2	IMPLEMENTED / E2E PENDING	fila IRQ1 + fallback de polling; 23 contratos do decoder
Page tables Konaet	IMPLEMENTED / VM E2E PASS	PML4 privados, CR3, NX/W^X, guardas, user-copy e 48+18 contratos
Scheduler de política	IMPLEMENTED / E2E PENDING	quantum, prioridades, RR, aging, sleep/block e 60 contratos
Threads preemptivas	IMPLEMENTED / VM E2E PASS	frame de 160 bytes, ring 0/CPL3, sleep/exit, XSAVE e 70 contratos
Ring 3/syscall/processos	IMPLEMENTED / VM E2E PASS M1-M4	PML4/CR3, tres ELF64, SYSCALL/SYSRETQ v2, VM, spawn/wait, thread create/exit, 100.000 ponteiros hostis e #PF contido
IPC Core I	IMPLEMENTED / VM E2E PASS, M5 PARCIAL	canais FIFO limitados, wait queues, direitos, handle transfer, backpressure e syscalls Ring 3 não bloqueantes
Workspace	PASS em host	36 contratos; estado volátil
Intelligence Seed	PASS em host	94 contratos; não é LLM
VMware end-to-end	PASS PARA M1-M4 + IPC CORE I	2 GiB/1 vCPU; IRQ0/iretq, ELF64, ABI v2, isolamento e IPC inicial confirmados

O manifesto registra **2.435 verificações PASS**: 912 estruturais, 247 de soberania, 210 do renderer, 94 de IA, 59 de arquitetura, 36 do workspace, 208 de memória, 37 do heap, 32 do boot protocol, 38 do clock, 154 de interrupções, 23 do decoder de input, 48 de paginação, 60 do scheduler, 70 de threads, 30 de syscall, 18 de processo, 18 de isolamento VM, 60 de ELF64 e 81 de IPC.

3. Cadeia completa do artefato atual

```
fontes Konaet + version.json + toolchain.lock.json
-> scripts/build.ps1
-> Zig/LLVM externo e fixado por hash
-> BOOTX64.EFI freestanding, sem biblioteca do host
-> FAT12 /EFI/BOOT/BOOTX64.EFI
-> ISO9660 + catálogo El Torito
-> verificadores PE/FAT/ISO e soberania
-> dezoito harnesses nativos
-> ISO + SHA-256 + manifesto de fontes/inputs/artefatos
```

Zig/LLVM e PowerShell/.NET são dependências externas declaradas de construção. Eles não são empacotados no EFI, mas impedem alegar toolchain soberana.

4. Cadeia de boot e separação de plataforma

```
firmware UEFI
-> platform_uefi.c localiza GOP e seleciona modo
-> obtém o mapa de memória
-> ExitBootServices
-> converte tipos EFI para tipos Konaet
-> monta ko_boot_context v1
-> valida magic, versão, tamanho, reservas e limites
-> ko_runtime_start(context)
-> arquitetura, memória, heap, clock, UI, workspace e IA
```

`platform_uefi.c` é o único módulo autorizado a incluir `uefi.h`. O runtime não conhece a ABL de firmware. O Konaet Boot Protocol usa magic `KOBT`, versão 2, contexto x86-64 de 88 bytes, campos reservados zerados, base/tamanho da imagem carregada e no máximo 256 descritores de memória.

Ainda falta um segundo produtor do protocolo em hardware de laboratório e um firmware/loader Konaet para eliminar a fronteira UEFI.

5. CPU, exceções e serial

O núcleo instala GDT própria, IDT com 256 portas e 32 stubs específicos de exceção. O frame de 160 bytes preserva RAX, RCX, RDX, RBX, RBP, RSI, RDI, R8–R15, vetor, error code, RIP, CS e RFLAGS. Vetores sem error code recebem zero explícito. Entradas não atribuídas conduzem a um caminho fatal.

COM1 é inicializada na porta 0x3F8 com timeout. Ausência do dispositivo não é rotulada como sucesso. O panic ainda precisa formatar todo o contexto, usar O double fault usa TSS/IST1 com pilha dedicada de 16 KiB. O panic ainda precisa formatar todo o contexto e ser validado por injeção física controlada.

6. Interrupções, timer e idle

O módulo de interrupções remapeia o PIC legado, programa o PIT para 100 Hz, instala IRQ0/vetor 32 e IRQ1/vetor 33 e envia EOI. Cada IRQ0 alimenta o clock monotônico. IRQ1 captura status e scan code em uma fila SPSC limitada a 31 bytes úteis; overflow é contado e nunca sobrescreve uma entrada pendente. Os contadores são `volatile` e saturam em vez de fazer wrap.

O runtime só executa `hlt` depois de observar o primeiro tick. Se a plataforma não entregar IRQ0, permanece em `pause`, evitando congelamento imediato. IRQ1 só é liberada depois que o controlador PS/2 é confirmado; se isso falhar, o runtime preserva polling como fallback.

Os modos estáticos de diagnóstico/informação encerram em `hlt`, sem busy-loop. Falhas de memória ou heap mostram panic e param de forma determinística. A entrada só aparece `LIVE` depois de resposta válida do controlador ou byte limpo do teclado; a ausência de PS/2 não é promovida a sucesso.

Faltam APIC, I/O APIC, timer por CPU, MSI/MSI-X, SMP, calibração física, medição de deriva e testes de interrupt storm.

7. Memória

O adaptador converte o mapa EFI para `ko_memory_descriptor`. O allocator aceita somente regiões utilizáveis, alinhadas e acima de 1 MiB; rejeita overflow; ordena e combina intervalos; reserva framebuffer, mapa original e arena do heap; aloca páginas contíguas com alinhamento; libera somente alocações exatas; e funde intervalos para reuso.

O heap atual é uma arena bump estática de 64 KiB. Ele mede uso, pico, falhas e alocações, mas não libera objetos individualmente.

As page tables Konaet são ativadas por CR3 com identity map de 2 MiB e divisão seletiva em páginas de 4 KiB. O kernel liga `EFER.NXE`, aplica headers `R--`, código `R-X`, dados `RW-`, rodata/unwind/reloc `R--`, rejeita `W+X` e instala guardas na página nula, IST de double fault, dois lados do heap e abaixo de pilhas de thread. Dois processos usam PML4 próprios, o mesmo VA com frames físicos distintos, troca de CR3 no IRQ0 e user-copy validado. Page fault CPL3 termina somente o infrator. Faltam copy-on-write, slab, quotas, ASLR e guardas para futuras threads dinâmicas.

O scheduler de política possui tabela fixa de 16 tarefas, quantum, oito prioridades, round-robin, aging contra starvation, sleep por deadline, block/wake e estado zombie. IRQ0 salva um frame de 160 bytes com GPRs, RIP/CS/RFLAGS/RSP/SS, seleciona o novo RSP e restaura a tarefa por iretq. Duas workers, em pilhas independentes de 16 KiB, executam sem yield, dormem por deadline e preservam valores MXCSR diferentes; uma terceira retorna e é transformada em zombie pelo IRQ0. Spinlock atômico, irqsave, bloqueio aninhado de preempção e XSAVE x87/SSE por slot estão ativos. Ainda faltam mutexes bloqueantes, futex, eventos/timers, APIC e SMP. Wait queues limitadas agora fundamentam o primeiro corte de IPC.

O loader ELF64 estático valida formato x86-64 little-endian, limites com overflow protegido, PT_LOAD, alinhamento, sobreposição, entry point e W^X. Ele mapeia código/dados/BSS e constrói stack inicial com argc, argv, envp e auxv. Três executáveis freestanding independentes são incorporados à image store do kernel; isso comprova execução ELF, mas ainda não é leitura por VFS, linker dinâmico ou cadeia geral de aplicações.

A ABI nativa v2 usa SYSCALL/SYSRETQ, frame kernel de 128 bytes e argumentos em RAX/RDI/RSI/RDX/R10/R8/R9; a ABI int 0x80 v1 permanece explicitamente versionada para compatibilidade. Estão implementadas consulta de ABI, exit/yield/clock/sleep, map/unmap/protect, spawn/wait, thread create/exit e handle close. O teste M4 cria processo com PML4/ELF próprios, coleta status 37, faz reap, cria thread com PML4 compartilhado e stack privada e preserva o kernel após 100.000 ponteiros hostis. Frames CPL3 suspensos são copiados para armazenamento privado por thread antes da troca CR3; não permanecem expostos à sobrescrita na única pilha TSS RSP0.

IPC Core I acrescenta canais com oito mensagens de até 64 bytes, FIFO, backpressure, filas wake-one, handles tipados/geracionais, atenuação de direitos e até dois handles transferidos por mensagem no núcleo. As syscalls 13-15 exercitam create/send/receive não bloqueantes em Ring 3. Bloqueio atômico, timeout/cancelamento, mutex PI, futex, sinais/eventos e timers ainda faltam para fechar M5.

8. Entrada, Control Center e workspace

Antes de ExitBootServices, o menu usa entrada textual EFI. Depois, o driver PS/2 consulta as portas 0x64/0x60, filtra erros e decodifica subconjuntos dos scan-code sets 1 e 2. A interface possui Overview, Arch/Hardware, Memory, Diagnostics/Events, Vision/Roadmap, Console, Intelligence e Files.

O workspace contém descritores compilados e navegação em RAM. Não é VFS, não lê disco e perde todo o estado no reset. O renderer escreve diretamente no framebuffer; ainda não há compositor, janelas, mouse, Unicode completo, acessibilidade, áudio ou aceleração GPU.

9. Intelligence Seed

A Intelligence Seed é um classificador determinístico de dez intenções. Um corpus embutido gera pesos inteiros Q4-range sobre 2.048 features. O roteador aceita somente comandos conhecidos e responde com buffers limitados e fatos do kernel. SHA-256 produz fingerprints e um log encadeado de quatro eventos.

Ela não é LLM, AGI, transformer, agente autônomo, memória conversacional ou prova causal. Faltam origem de arquivos segura, parser GGUF, tokenizer, operadores tensoriais, runtime transformer, isolamento em userland e avaliações de segurança antes de qualquer ferramenta com efeito colateral.

10. Segurança e modelo de confiança

O EFI possui zero imports de bibliotecas do host. Includes do kernel resolvem para headers Konaet; UEFI fica confinado ao adaptador. O build bloqueia tokens de link proibidos e artefatos de emulador na release ativa.

O PE real também é auditado seção por seção: alinhamento de página, limites em `SizeOfImage`, presença das classes `R-X`, `RW-` e `R--`, e ausência de `W+X`.

Isso ainda não equivale a um sistema seguro para uso diário. Os milestones M1-M4 e IPC Core I foram comprovados, mas faltam IPC bloqueante, sinais, uma ABI de userland completa, capabilities, IOMMU, entropia, DRBG, cofre de chaves, ACL, ASLR, boot verificado/medido, assinatura, atualização atômica, rollback, fuzzing e auditoria independente.

11. Build, testes e reprodução

Comando atual:

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\build.ps1
```

Saídas principais:

```
build/esp/EFI/BOOT/BOOTX64.EFI
build/efiboot.img
dist/konaet-os-0.3.4-x86_64-uefi.iso
dist/konaet-os-0.3.4-x86_64-uefi.iso.sha256
dist/konaet-os-0.3.4-manifest.json
build/tests/*-tests.exe
```

Os testes de host comprovam contratos determinísticos e limites, não execução privilegiada real. Um boot físico declarado deve validar GOP, ExitBootServices, GDT/IDT, exceções, PIC/PIT, clock, teclado e estabilidade.

12. Artefatos auditados

Artefato	Bytes	SHA-256
BOOTX64.EFI	199.680	7b373b5cb38b8d3fd440b5746dd9b986c3a98bbcb402c89e09b662add096998
efiboot.img	1.474.560	6e4efcc0b0a563cf2a9156abd12a8aec1f5407df67f93b61c01fd7781e3dd1c
ISO	1.521.664	e7ec43177f6c06790b3a563004ccc0b888abbe71c79927e7240fe8eb229262d2
Manifesto	40.596	c2dc1c5be2a7bec55a6abb7e47236e22d2c08517a307dcab4564270f4dfb9d11

O `.sha256` e o manifesto devem ser consultados novamente depois de qualquer alteração de fonte ou build.

As datas de criação, modificação e efetividade ISO9660 são `2026082600000000`, derivadas de `version.json`; o gate estrutural também confere a data curta dos registros de diretório.

13. Validação VMware isolada

A bancada externa foi configurada em UEFI com 2 GiB de RAM, 1 vCPU e a ISO como `cdrom-image`. O disco virtual existente de 10 GiB, NAT e áudio permaneceram sem uso porque o Konaet ainda não possui esses drivers. COM1 foi ligada a um arquivo de evidência. Nenhum disco físico, partição, BCD, TPM ou firmware do host foi alterado.

Depois da ativação manual de SVM/AMD-V, o boot registrou:

```
KONAET IRQ0 XSAVE PREEMPTION ARMED
KONAET PREEMPTIVE THREAD START
KONAET IRQ0 FRAME SWITCH
KONAET IRQ0 IRETQ XSAVE SLEEP EXIT SELF-TEST PASS
```

O Control Center abriu em 800 x 600 e exibiu `XSAVE PREEMPT LIFE: LIVE`. Esse rótulo exige os contadores do autoteste, portanto confirma este marco na VM. A captura serial canônica confirma PML4/CR3 distintos, três ELF64, `SYSCALL/SYSRETQ v2`, VM map/protect/unmap, spawn/wait/reap, thread create/exit, IPC limitado em Ring 3, 100.000 syscalls com ponteiro hostil e um `#PF` de A contido enquanto B e o kernel continuam; isso não substitui boot em hardware físico nem valida disco, filesystem ou rede.

14. Diagnóstico e recuperação

O estado atual oferece COM1 mínima e telas de panic. Ainda faltam log estruturado, ring buffer, timestamps físicos validados, stack trace, símbolos no alvo, crash dump, tracing, métricas persistentes, modo seguro e

recovery shell. Falhas críticas devem parar de forma determinística até existir recuperação comprovadamente segura.

O diagnóstico separado do host encontrou eventos de esgotamento de commit por instâncias antigas de `codex.exe`, não uma VM Konaet. Nenhum guest estava em execução. Detalhes e medidas de contenção estão em `docs/HOST_STABILITY_DIAGNOSIS_2026-08-26.md`.

15. Arquitetura de uso geral desejada

```
firmware Konaet
-> loader + KBP
-> kernel: CPU, IRQ, tempo, MMU, scheduler
-> processos ring 3 + syscalls + capabilities
-> barramentos e drivers
-> bloco + VFS + filesystem
-> sockets + rede + firewall
-> init + serviços + login + pacotes
-> compositor + desktop + aplicações
-> atualização A/B + rollback + recovery
-> observabilidade + qualificação
```

O Control Center deve migrar de ring 0 para processo não privilegiado. Drivers complexos devem operar com capacidades mínimas e falhar sem derrubar o kernel.

16. Gates obrigatórios G00–G18

Gate	Entrega	Estado atual
G00	especificação, build, manifesto, ABI e reprodução	IN PROGRESS
G01	boot, KBP e evidência física	IMPLEMENTED / E2E PENDING
G02	exceções, IRQ, tempo, teclado, APIC e SMP	IN PROGRESS
G03	memória virtual, NX/RO/W^X e heap recuperável	IN PROGRESS: PML4 privado, CR3, user-copy e #PF CPL3 contido; COW, ASLR e heap recuperável pendentes
G04	threads, scheduler, sincronização e preempção	IN PROGRESS: IRQ0/iretq, spinlock, sleep/exit, XSAVE e wait queues validados; mutex PI, futex e SMP pendentes
G05	ring 3, processos, syscalls, IPC e init	IN PROGRESS: M1-M4 e IPC Core I PASS com processos, PML4/CR3, ELF64, ABI v2, canais limitados e Ring 3; IPC bloqueante, sinais e init de serviços pendentes
G06	capabilities, identidade, entropia e sandbox	NOT IMPLEMENTED
G07	ACPI, PCIe, MSI, IOMMU, USB e energia	NOT IMPLEMENTED
G08	bloco, NVMe/AHCI, flush, TRIM e erro	NOT IMPLEMENTED
G09	VFS, initramfs e filesystem recuperável	NOT IMPLEMENTED
G10	Ethernet, IP, TCP/UDP, DNS/DHCP e firewall	NOT IMPLEMENTED
G11	libc/runtime, shell, serviços, login e pacotes	NOT IMPLEMENTED
G12	compositor, janelas, input, áudio e acessibilidade	NOT IMPLEMENTED
G13	assinatura, atualização A/B, rollback e backup	NOT IMPLEMENTED
G14	logs, panic, dump, tracing e diagnóstico	NOT IMPLEMENTED

Gate	Entrega	Estado atual
G15	assembler, linker, compilador e self-hosting	NOT IMPLEMENTED
G16	firmware Konaet em placa recuperável	NOT IMPLEMENTED
G17	plataforma física controlada e reproduzida	NOT IMPLEMENTED
G18	qualificação e benchmarks comparativos	NOT IMPLEMENTED

17. Ordem de construção

1. validação física de IRQ0, IRQ1, fila de input e timer;
2. APIC, timer por CPU, validação TSS/IST e contexto de panic;
3. ampliar faults recuperáveis, COW, ASLR, guardas dinâmicas e heap recuperável;
4. completar M5 com IPC bloqueante, mutex PI, futex, eventos/timers e testes prolongados;
5. sinais, initramfs, capabilities e expansão do userland;
6. ACPI, PCIe, MSI/IOMMU e drivers fundamentais;
7. bloco, NVMe/AHCI, VFS e filesystem;
8. rede e firewall deny-by-default;
9. userland, login, serviços, shell e pacotes;
10. compositor, desktop, acessibilidade, áudio e aplicações;
11. atualização assinada, A/B, rollback, backup e recovery;
12. observabilidade, fuzzing, testes prolongados e auditoria;
13. toolchain, firmware e hardware Konaet em laboratório dedicado;
14. benchmarks publicados e qualificação final.

18. Critério de superioridade

“Superior a todos” não é uma propriedade técnica única. Cada alegação precisa de eixo, versões, hardware, configuração, código do benchmark, resultados brutos, dispersão e no mínimo cinco repetições. Eixos válidos incluem tempo de boot, RAM, tamanho da base confiável, latência do scheduler, throughput de storage/rede, isolamento, recuperação, energia, estabilidade e IA.

O projeto pode vencer um eixo e continuar incompleto em outros. Nenhum painel, percentual visual ou quantidade de testes substitui essa medição.

19. Definição final de pronto

Uma release só será `END-TO-END READY` quando:

- todos os gates G00–G18 estiverem PASS;
- construir, instalar, atualizar e recuperar a si própria;
- inicializar repetidamente em hardware físico declarado;
- possuir processos isolados, storage, filesystem, rede, drivers e userland;
- conter falhas de processos e drivers sem corromper o kernel;
- sobreviver a testes de energia, atualização interrompida e longa duração;
- eliminar ou declarar formalmente cada fronteira externa;
- publicar resultados que sustentem cada alegação de superioridade.

Até lá, o estado correto é **kernel autoral executável em evolução**, com evidência crescente e limites explícitos.

20. Mapa de fontes

Área	Arquivos principais
Plataforma	platform_uefi.c , uefi.h
Boot	boot.c , boot.h
Runtime	runtime.c , runtime.h
CPU	arch.c , arch.h , system.c , system.h
Tempo/IRQ	clock.c , clock.h , interrupt.c , interrupt.h
Memória/processos	memory.c , heap.c , paging.c , process.c , vm.c , elf.c e respectivos headers
Scheduler/threads	scheduler.c , scheduler.h , thread.c , thread.h , thread_switch.S
Syscalls	syscall.c , syscall.h , syscall_entry.S
Entrada	input.c , input.h
UI	ui.c , ui.h
Workspace	workspace.c , workspace.h
IA/Hash	ai.c , ai.h , hash.c , hash.h
Build	scripts/build.ps1 , New-KonaetIso.ps1
Gates	verify.ps1 , verify-sovereignty.ps1
Testes	tests/*_harness.c
Auditoria integral	docs/FULL_FOLDER_AUDIT_2026-08-26.md
Estabilidade do host	docs/HOST_STABILITY_DIAGNOSIS_2026-08-26.md

Documentos de apoio: ARCHITECTURE.md , MEMORY.md , INTELLIGENCE.md , THREAT_MODEL.md , SOVEREIGNTY.md , KONAET_BOOT_PROTOCOL.md , PROCESS_ISOLATION.md , ELF_LOADER.md , SYSCALL_ABI.md , END_TO_END_COMPLETION.md , ROADMAP.md e o relatório da release.